scanner_blindness_investigation_20260826

BOB-191 — Fail-open scanner blindness to Go: measured investigation

Revision: 1 **Last modified:** 2026-08-26T00:00:00Z **Status:** investigation complete — INVESTIGATION ONLY, no source modified **Label:** (T11/002-user-owned-downloads - milos85vasic - ? - xhigh) **Scope:** BOB-191 (false null / Go), BOB-189 (false match / SSRF guards), BOB-192 (6 ratcheted findings) **Authority:** §11.4.201(1)(6)(7)(b), §11.4.6, §11.4.224(E), §11.4.245, §11.4.252

---

1. What the scanner is, and where it runs

Layer	Path	Role
Invariant 39 (caller)	`scripts/pre_build_verification.sh:1481-1544`	Iterates `DANGER_ROOTS=(download-proxy/src plugins scripts qBitTorrent-go frontend/src)`, calls the gate once per root. ADVISORY / non-blocking.
Gate (the scanner)	`constitution/scripts/gates/cm_dangerous_combination_fail_closed.sh` (813 lines)	Enumerates source files, applies analysers, emits PASS/FAIL.

`scripts/pre_build/check_cm_no_fail_open_skip.sh` does **not** exist on any branch (`git log --all --diff-filter=A` returns empty); it exists only in the uncommitted worktree `.claude/worktrees/agent-`

a5cb87da3e518c991/. The scanner in force today is the gate above. *(Control needle: the same grep -rl found CM-PLUGIN-COUNT in three files, so the instrument was not blind when it returned zero.)*

2. Exactly what it parses vs. what it skips — with the lines

Enumeration (cm_dangerous_combination_fail_closed.sh:243)

```
exts="${DANGEROUS_COMBO_EXT:-py go rs c cc cpp h hpp java cs js ts jsx tsx php rb}"
```

go **is** in the list. .go files **are** enumerated. This is *not* a scope hole.

Analysis — the complete analyser set:

Analyser	Location	Applies to
(A1/A2/C) Python ast — swallow / silent-default / contextlib.suppress	:537 — case "$f" in *.py) py_files+=("$f") ;;	**.py only**, by explicit extension test
(A) catch (...) { } empty-body text scan	:772 — grep -nE 'catch[[:space:]]*\([^)]*\)[[:space:]]*\{'	every file, but keyed on the literal token catch
(B) credential = x || "literal" / or "literal"	:794	every file, but keyed on ||/or + quoted literal

The decisive line is :537: the structural analyser is gated on *.py. Every non-Python file falls to the two text matchers at :766-804. **Go has no catch keyword and no ||-as-nil-coalescing idiom, so neither matcher can fire on Go by construction.** The blindness is a *matcher-coverage* hole, not a scope hole.

3. Control needle — both arms (§11.4.201(7)(b))

Needles planted on **copies in scratch**, never in the repo. Run through the identical path (bash <gate> --root <dir>).

Arm	Planted	Gate rc	Verdict	Finding lines
Python (except Exception:	2	**1**	FAIL	**2 — SEEN**

Arm	Planted	Gate rc	Verdict	Finding lines
`pass`, `except: return None`)				
TypeScript (`catch (e) { }`)	1	**1**	FAIL	**1 — SEEN**
Java (`catch (Exception e) { }`)	1	**1**	FAIL	**1 — SEEN**
Go (empty `if err != nil {}`, `_ =`, `, _ :=`, bare `recover()`, comment-only err block, credential-to-literal)	**6**	**0**	**PASS**	**0 — NOT SEEN**
Rust (`let _ =`, `Err(_) => {}`)	2	0	PASS	0 — NOT SEEN
Ruby (`rescue StandardError` + comment)	1	0	PASS	0 — NOT SEEN
C (ignored return, empty `if (...) {}`)	2	0	PASS	0 — NOT SEEN

The Go needle is **genuinely valid Go**, not a strawman: `go build ./...` rc=0, `go vet ./...` rc=0, `gofmt -e` reports no syntax errors (go1.26.2). Note that `go vet` is silent on these shapes too — this class needs a dedicated arm.

3a. The discriminator: matcher-hole vs scope-hole

Same bytes, only the extension changed:

File	Enumerated?	Gate output
`needle.zig` (ext **not** in list)	no	`⏭ SKIP — topology_unsupported: no source files under scan`

File	Enumerated?	Gate output
`needle.go` (ext **is** in list)	yes	`PASS — no ... anti-patterns found`

This is the core defect. The gate already owns an honest-blindness mechanism — `SKIP — topology_unsupported` — and it works. Go routes *around* that mechanism by being enumerated: it is counted as scanned, produces zero, and prints a green PASS. A scope hole announces itself; this matcher hole does not. The matcher hole is the **worse** of the two defects for exactly that reason.

4. Quantified unscanned Go surface — with exclusions stated (§11.4.224(E))

Exclusion list applied (the scanner's own `DANGEROUS_COMBO_EXCLUDE` default, nothing added): `.git node_modules vendor .venv __pycache__ scripts/gates out build dist`

Measure	Count
`.go` under `qBitTorrent-go/`, **no** exclusions	**106**
`.go` under `qBitTorrent-go/`, **with** the scanner's prune list	**106**
Total Go LOC in that set	**20,389**
Production (non-`_test.go`) files / LOC	**52 / 6,858**
`_test.go` files	54

A == B (106 == 106). `find qBitTorrent-go -type d -name vendor` returns empty — there is no vendored Go tree, so the exclusion list removes nothing and no unstated exclusion could have manufactured a clean number.

Blind vs analysable inside invariant 39's own roots:

Root	Blind-extension files (go/rs/rb/c/h)	Analysable files (py/ts/tsx/js/jsx)
`download-proxy/src`	0	22
`plugins`	0	69
`scripts`	0	5
`qBitTorrent-go`	**106**	**0**
`frontend/src`	0	66

`qBitTorrent-go` is **100% blind**: every file it contains is of an extension no analyser covers. Its PASS is a pure false null. Invariant 39 nonetheless counts it toward `DANGER_SCANNED` and, when all roots pass, prints *"no fail-open anti-pattern across 5 first-party source root(s)"* — the blind root is presented as one of five clean ones.

4a. A second, distinct hole at the caller layer

`cmd/boba-ctl/` — **4 `.go` files, 947 LOC** — is **absent from `DANGER_ROOTS` entirely**. That *is* a scope hole (§11.4.224(E)), separate from the matcher hole, and it covers the container orchestrator: a shell-exec + mutation surface, i.e. a textbook §11.4.252 dangerous combination. Not scanned even in principle.

5. Real fail-open constructs in LAN-facing Go — leads read as lines

Probe was itself needle-validated (it saw the planted needle; it stayed clean on a negative-control file with a correctly-propagated error). **Its honest limit:** it matches only bare `if err != nil {` openers, not `if err := f(); err != nil {`.

No empty `if err != nil {}` blocks exist in production Go — that shape is genuinely absent. The real findings are discarded-error shapes in the credential path:

F1 — `qBitTorrent-go/internal/jackettapi/credentials.go:234-253` (credential DELETE) — most serious

```
234   if err := d.Repo.Delete(name); err != nil {   // checked,
fails closed
239   // 2) .env delete (best-effort — DB is canonical at this
point).
240   _ = envfile.Delete(d.EnvPath, []string{ name+"_USERNAME",
name+"_PASSWORD", name+"_COOKIES" })
246   // 3) Jackett-side delete (best-effort).
249       _ = d.Jackett.DeleteIndexer(id)
253   w.WriteHeader(http.StatusNoContent)          //
unconditional success
```

If the `.env` delete fails, the plaintext credential variables (named here, values never read — §11.4.10) **remain on disk**, and the API answers **204 No Content**: the caller is told the credential is gone while the secret persists. Likewise a failed `DeleteIndexer` leaves Jackett configured against a deleted credential. "Best-effort" is a comment, not a mechanism; the 204 asserts a state transition the

code never verified. Capabilities combined: credential-access + mutation + irreversible + external-side-effect — four, where §11.4.252 requires ≥ 2.

F2 — `credentials.go:166-176` (credential UPSERT compensating rollback)

```
166   if err := envfile.Upsert(d.EnvPath, envKV); err != nil {
167       // Compensating rollback.
169       _ = d.Repo.Delete(body.Name)
171       _ = d.Repo.Upsert(prior.Name, prior.Kind, ...)
174       writeJSONError(w, 500,
"env_write_failed_db_rolled_back", err.Error())
```

The error code **asserts** `db_rolled_back`, but the rollback's own error is discarded. If the rollback also failed, the API still reports it succeeded — a §11.4 bluff at the API-response layer and a §11.4.253 compensation-without-verification.

F3 — benign, stated so it is not inflated

`client.go:140,153 _ = resp.Body.Close()`, `credentials.go:274` / `openapi.go:35` / `runs.go:112 _ = json...Encode/Write`, `runs.go:43 _ = json.Unmarshal`, `autoconfig.go:229 _ = deps.Client.WarmUp()` — idiomatic Go cleanup / already-committed response writes. **Not** findings; listed so the count is honest in both directions.

F4 — the fail-CLOSED control (golden-FALSE material)

`qBitTorrent-go/internal/jackettapi/auth_middleware.go:45-66` is a correct fail-closed guard: non-GET/HEAD/OPTIONS requires the header, compared with `subtle.ConstantTimeCompare`, else 401 + return. It matches BOB-203's measurement that 7189 writes are 401-gated. **Any future Go arm must not flag this.**

6. BOB-189's mirror — independently confirmed

`download-proxy/src/api/routes.py:1122 _is_safe_fetch_url(url) -> bool` is a textbook fail-**closed** SSRF guard (rejects non-http(s), missing host, DNS failure, and any loopback/RFC-1918/link-local/`169.254.169.254`/multicast address). Its **only** call site, `:1476`, reads `if not _is_safe_fetch_url(url):` → refuse.

The scanner flags `:1143`, `:1155`, `:1166` — every one a `return False` that *is* the refusal — under shape (A2) *"silent default return"*. Acting on the finding means making an SSRF guard stop returning `False`. Confirmed, not taken on report: **a gate that instructs you to delete a security control.**

7. Verdict — one design problem, two remediations

One primitive defect, two symptoms (§11.4.250 / §11.4.245):

> The scanner classifies by **local syntactic shape** and never consults the **semantic role** of the construct — neither what the caller does with the value, nor what error-handling idiom the language actually uses.

- BOB-189 = shape matched without the **caller's** context → false MATCH.
- BOB-191 = a shape from a **different language family** searched for → false NULL.

The gate's own header already concedes the primitive: *"PROVING that a code path genuinely COMBINES >= 2 dangerous capabilities … requires real data-flow / control-flow analysis this gate CANNOT honestly claim,"* and calls itself *"the LITERAL, NAMED, structurally-precise HALF."* The two bugs are the two edges where that half fails. What is missing is not the acknowledgement — it is the **propagation of that acknowledgement into per-language honesty**.

Load-bearing prediction: **bolting a naive Go regex arm on will immediately manufacture BOB-189-class false positives in Go** — `if err != nil { return false }` inside a Go validator is fail-*closed* and would be flagged. Fixing BOB-191 as layer N+1 without addressing the primitive reproduces BOB-189 in a new language (§11.4.250: the tower is the diagnostic).

They should stay **separate tracker items** (§11.4.214 — different fixes, and merging hides one behind the other), but be **designed together**.

8. Remediation direction (not implemented — not authorised this session)

1. **Analyser registry with an UNANALYSED verdict (the single structural fix).** Map extension → analyser. A file whose extension has **no registered analyser** is reported `UNANALYSED`,

never silently passed; a root with zero analysable files SKIPs with reason instead of PASSing. Then `qBitTorrent-go` reads *"106 files UNANALYSED — no Go analyser"* rather than green. This alone converts BOB-191's false null into an honest §11.4.6 gap, is language-agnostic, and also covers rust/ruby/C measured blind above. **Do this before any Go arm.**
2. **Go arm, call-site-aware from day one** (`go/ast` via `gofmt`-grade parsing, or adopt `errcheck`/`staticcheck` SA-checks rather than hand-rolling regex): empty/comment-only `if err != nil {}`, `_ = <err-returning call>` on dangerous-capability calls, `, _ :=` discards, `recover()` without re-panic. Ship with `auth_middleware.go:45-66` as a **golden-FALSE fixture** (§1.1) so the fail-closed/fail-open discrimination is falsifiable from the start.
3. **BOB-189:** make shape (A2) call-site-aware — a `return False` whose call sites are all refusal-shaped (`if not f(...)`) is fail-closed. Add `_is_safe_fetch_url` as the golden-FALSE fixture, as BOB-189's acceptance requires.
4. **Close the caller-layer scope hole:** add `cmd/boba-ctl` to `DANGER_ROOTS`, or replace the hand-maintained root list with a declared inclusion manifest (§11.4.251 — no hand-maintained gate list).
5. **BOB-192 note:** its 6 ratcheted findings and BOB-189's 6 false matches are *different sets*; per the reviewer's MINOR-1 a count baseline absorbs a one-out-one-in swap, so track the finding **set**, not the count — especially while items 1-3 change what the scanner can see.

9. Anti-bluff provenance

- Every reported zero carries a class-matched needle proving the path can see (§11.4.201(7)(b)); the Go zero is reported as **blindness**, never as clean.
- The `grep` that found no `NO-FAIL-OPEN` gate was needle-checked against a known-present token before its zero was believed.
- **Own instrument bug caught and corrected:** a first pass extracted the verdict with `grep -oE 'PASS|FAIL|SKIP'`, which matched `FAIL` inside the gate's own name `...FAIL-CLOSED` and mislabelled three PASSes. Re-run keyed on the exit code and the `/ /⏭` line. Recorded rather than quietly fixed (§11.4.201(9)).
- Counts are leads; §5 findings are read lines with file:line.
- No repo file modified; all needles on copies under the session scratchpad.
- Credentials referenced by variable NAME only, never by value (§11.4.10).